

CHANGELOG — Kamai Backend OMS

All notable changes to this project will be documented in this file.

Format follows [Keep a Changelog](#).

[Unreleased]

Changed

- **Authentication Architecture** — Migrated from AuthKey SMS OTP to Firebase Phone Authentication (ADR-009)
 - Client is the **Kamai web app (browser-based)**; Firebase JS SDK (**firebase/auth**) handles OTP on the frontend
 - Backend no longer generates, sends, stores, or verifies OTPs
 - Backend now verifies Firebase ID Tokens using Firebase Admin SDK
 - OTP lifecycle (generation, delivery, verification, retry) is entirely managed by Firebase / Google
 - New env vars required: **FIREBASE_PROJECT_ID**, **FIREBASE_CLIENT_EMAIL**, **FIREBASE_PRIVATE_KEY**
 - **firebase-admin** npm package to be added as dependency in Action 1
 - Baker accounts will be keyed by **firebaseUid** (Firebase UID) in addition to phone number
 - Access + refresh tokens delivered as **HttpOnly cookies** (browser-safe)
 - No OTP-related Redis keys or database tables will be created
 - Twilio / SMS provider removed from auth scope (Twilio retained only for WhatsApp order notifications)
-

v0.9.0 — 2026-07-26

Added

- **Action 9 — Edit Order**

- **PUT /api/orders/:orderNumber** — Endpoint to fully replace editable fields on an existing order.
 - Checks customer phone number changes for uniqueness conflicts (**409 Conflict**).
 - Recalculates **balanceDue** and **paymentStatus** on server-side dynamically without modifying the immutable **PaymentLedger**.
 - Rejects edits for **DELIVERED** and **CANCELLED** orders.
 - Implemented under a single database transaction with robust validation and **ORDER_UPDATED** auditing.
-

v0.8.0 — 2026-07-26

Added

- **Action 8 — Record Balance Payment**

- **PATCH /api/orders/:orderNumber/payment** — API endpoint to record financial payments against an order.
 - Implemented **FinanceService** to track payments inside the new **PaymentLedger** table.
 - Introduced **PaymentStatus** (**UNPAID**, **PARTIALLY_PAID**, **PAID**) dynamically assigned on order creation and updated upon payment.
 - Included strict balance validation and robust concurrency control via Prisma **\$transaction**.
 - Added **PAYMENT_RECORDED** audit logs.
-

v0.7.0 — 2026-07-26

Added

- **Action 7 — Update Order Status**

- **PATCH /api/orders/:orderNumber/status** — API endpoint to advance orders through the production lifecycle.

- Built `StatusValidationService` to strictly enforce the state machine (`PENDING` -> `CONFIRMED` -> `IN_PROGRESS` -> `READY` -> `DELIVERED`).
 - Added custom `INVALID_ORDER_STATUS_TRANSITION` error code bridging to `409 Conflict`.
 - Executed status updates via Prisma `$transaction` avoiding concurrency issues.
 - Emitted `ORDER_STATUS_UPDATED` audit log on successful transition.
 - Dashboard cache invalidation on order mutation.
-

v0.6.0 — 2026-07-26

Added

- **Action 6 — View Order Details**

- `GET /api/orders/:orderNumber` — fetching full details of a specific order
 - Integrated `NotFoundError` handling for missing or soft-deleted orders
 - Validated parameter schemas with Zod and provided comprehensive Swagger documentation
-

v0.5.0 — 2026-07-26

Added

- **Action 5 — Order History**

- `GET /api/orders` — retrieval of order pipeline with dynamic filtering
- Supports filtering by `status`, `search` (customer name, phone, orderNumber), and `deliveryDate` (or `from/to` range)
- Added multi-sorting (default: `deliveryDate` ASC, then `createdAt` DESC)
- Implemented server-side pagination (`page`, `limit`)
- Added Prisma compound indexes (`[bakerId, status]`, `[bakerId, deliveryDate]`, `[bakerId, createdAt]`, `[bakerId, orderNumber]`)
- Swagger documentation for the new query parameters and response schema

v0.4.0 — 2026-07-26

Added

- **Action 4 — Create New Order**

- `POST /api/orders` — core order creation workflow
 - `Customer` model with `upsert` logic inside Prisma `$transaction`
 - Extended `Order` model (fields: `customerId`, `orderNumber`, `category`, `weight`, `flavour`, `advancePaid`, `referencePhoto`, `deletedAt`)
 - Unique `orderNumber` generator (`OrderNumberService`)
 - Stubs for `AuditService` and `CacheService`
 - Updated `OrderStatus` enum to `PENDING`, `CONFIRMED`, `IN_PROGRESS`, `READY`, `DELIVERED`, `CANCELLED`
-

v0.3.0 — 2026-07-26

Added

- **Action 3 — Summary Dashboard**

- `GET /api/dashboard/summary` — returns daily operational metrics and orders list
- Dashboard Module (`src/modules/dashboard`)
- `Order` model: `bakerId`, `deliveryDate`, `status`, `totalPrice`, `balanceDue`
- `OrderStatus` enum: `DRAFT`, `IN_PRODUCTION`, `READY`, `DELIVERED`, `CANCELLED`
- Auth Middleware: `authenticate` plugin (`src/plugins/authenticate.ts`)
- Prisma migration: `20260725195702_add_order_model`

Changed

- `FastifyInstance` typed with `.authenticate` decorator

- Currency fields (`totalPrice`, `balanceDue`) implemented as `Int` (smallest currency unit, e.g., paise) to avoid floating point math errors
-

v0.2.0 — 2026-07-26

Added

- **Order Management - Cancel Order (Action 10)**
 - Implemented `DELETE /api/orders/:orderId` endpoint to soft-delete an order (status = CANCELLED, deletedAt = NOW()).
 - Protected terminal order states (DELIVERED, CANCELLED) with 409 Conflict.
 - Implemented `ORDER_CANCELLED` audit log.
- **Customer Directory & CRM Metrics (Action 11 & 12)**
 - Extracted customer upsert logic to `CustomersService`.
 - Automatically recalculates `totalOrders`, `lifetimeValue`, and `lastOrderDate` for CRM on order Create, Edit, Cancel.
 - Excluded `CANCELLED` orders from LTV computations.
 - Implemented `GET /api/customers` endpoint for paginated, searchable, and sortable customer lists.
 - Dynamically calculates `outstandingBalance` per customer across unpaid orders efficiently.
 - Created required `@@index` on `Customer` and `Order` models for efficient aggregation.
- **Customer Profile (Action 13 & 14)**
 - Implemented `GET /api/customers/:customerId` to retrieve a complete customer CRM profile.
 - Includes isolated tenant verification, dynamic `outstandingBalance` calculation, and a paginated lightweight order history.
 - Added new CRM fields: `notes` and `preferredDeliveryTime` to `Customer` schema.
 - Implemented `PUT /api/customers/:customerId` for editing customer profile.
 - Added strict tenant-scoped duplicate phone validation (`409 Conflict`).
 - Added `CUSTOMER_UPDATED` transaction-wrapped audit events recording precisely which fields changed.

- Created `@@index([customerId, createdAt])` on `Order` for optimal profile loading.

- **Calendar View (Action 15)**

- Implemented `GET /api/dashboard/calendar` to aggregate order data by delivery date.
- Supports `month` (default) and `week` views with ISO-8601 week boundaries.
- Generates continuous day arrays mapped to status (`pending`, `confirmed`, `inProgress`, `ready`, `delivered`) avoiding raw SQL for `balanceDue` and `status`.
- Excludes `CANCELLED` orders from production metrics payload.

- **Investment / Expense Ledger (Action 16)**

- Added `Investment` model to `schema.prisma` with Decimal precision for `quantity` and integer for `pricePerUnit`.
- Implemented `POST /api/investments` to log material purchases, calculating `totalCost` on write for optimal read performance.
- Implemented `GET /api/investments` with date range and pagination support, dynamically aggregating `totalExpense`.
- Implemented `DELETE /api/investments/:entryId` via soft deletes (`deletedAt`).
- Added `INVESTMENT_CREATED` and `INVESTMENT_DELETED` audit events.
- Fixed a global route plugin encapsulation bug by removing the `fp` wrapper from route modules (`auth`, `dashboard`, `orders`, `investments`).

- **Billing & Subscription (Action 17)**

- Added `SubscriptionStatus` (`PENDING`, `PAUSED`, `EXPIRED`) and `SubscriptionPlan` enums to `schema.prisma`.
- Extended `Baker` model with trial and Razorpay subscription tracking fields.
- Initialized trial dates (`createdAt + 90 days`) for all existing bakers via migration script.
- Designed an abstract `PaymentGateway` interface and implemented `RazorpayGateway` using the official `razorpay` SDK.
- Created `GET /api/billing/status` to fetch trial days remaining, plan details, and auto-renew status.
- Created `POST /api/billing/create-subscription` which requests a mandate from Razorpay, validates subscription conflicts (e.g. 409 if already `ACTIVE` or `PENDING`), updates the `Baker` status, invalidates cache, and logs `SUBSCRIPTION_CREATED`.

- **Razorpay Webhook Processing (Action 18)**

- Implemented `POST /api/webhooks/razorpay` to process Razorpay callback events natively and securely.
- Added `fastify-raw-body` plugin strictly scoped to the webhook route to enable HMAC SHA-256 signature verification matching Razorpay specs.
- Added `WebhookEvent` model for reliable, persistent idempotency checks (preventing duplicate billing operations) and `BillingHistory` ledger to log all billing actions.
- Implemented transactional update flow translating Razorpay events (`subscription.activated`, `subscription.charged`, etc.) to Kamai `SubscriptionStatus` states (`ACTIVE`, `PAUSED`, `CANCELLED`).
- Added `WebhookProcessor` interface with `RazorpayWebhookProcessor` implementation, keeping the generic webhook controller detached from Razorpay specifics.

• Manage UPI Settings (Action 19)

- Expanded `Baker` model in `schema.prisma` with payment collection fields: `upiId`, `merchantName`, `preferredApps`, `defaultCollectionMethod`, and `dynamicQrEnabled`.
- Introduced `CollectionMethod` enum to restrict method types to `UPI` or `QR`.
- Created `PaymentSettingsService` completely decoupled from generic Baker profile operations, handling transaction-safe updates and `UPI_SETTINGS_UPDATED` audit events.
- Created `PUT /api/baker/upi-settings` endpoint featuring strict `zod` validation including regex-based VPA format checking and a predefined `SUPPORTED_PAYMENT_APPS` array.

• Upload Business Assets (Action 20)

- Integrated `@supabase/supabase-js` to securely generate signed URLs for direct-to-storage client uploads, keeping the backend free of large file streams.
- Implemented `StorageProvider` abstraction and `SupabaseStorageProvider` ensuring storage vendor flexibility.
- Created two explicit endpoints: `POST /api/uploads/signed-url` to generate upload tokens and `POST /api/uploads/confirm` to safely write `logoPath` or `fssaiDocumentPath` to the database only after storage object existence verification.
- Added strict MIME type and upload category validation mapped to structured Supabase paths (e.g. `{bakerId}/logo/{uuid}.png`).

• WhatsApp Notifications (Action 21)

- Created `POST /api/notifications/whatsapp` endpoint to generate dynamic, pre-filled WhatsApp deep links (`wa.me`) without needing paid Meta

APIs.

- Architected the solution using **WhatsAppTemplateEngine**, **MessageFormatter**, and **DeepLinkGenerator** to cleanly decouple message construction, formatting, and delivery transport.
- Supported templates: **ORDER_CONFIRMATION**, **PAYMENT_REMINDER**, **READY_FOR_PICKUP**, **RECEIPT**, and **THANK_YOU**.
- Enforced strict validation: requires valid customer phone and automatically manages UPI instructions inclusion based on baker configuration and remaining balance due.
- Logs **WHATSAPP_LINK_GENERATED** audit event.

- **Baker Profile & Settings (Action 22)**

- Implemented **GET /api/baker/profile** providing a read-only, consolidated DTO of the Baker's business details, payment configuration, subscription, and FSSAI verification status.
- Upgraded **StorageProvider** interface with **getSignedReadUrl** allowing short-lived read access securely to private Supabase storage.
- Created **BakerProfileMapper** to cleanly decouple database structure from the structured presentation-layer DTO requirements.
- Handled automated logic like trial-days-remaining calculations directly within the profile mapper.

- **Support & Help Desk (Action 23)**

- Created **POST /api/support/chat** endpoint to construct a pre-filled WhatsApp deep link (**wa.me**) for instant support inquiries.
- Implemented **SupportMessageFormatter** to separate template creation from transport mechanisms, formatting diagnostic context (Business Name, Owner, Issue Type, Subscription Status, App Version).
- Dynamically resolved App Version from **package.json** with fallback handling.
- Reused **DeepLinkGenerator** for URL encoding and phone number normalization.
- Configured strict validation for **SupportIssueType** enum and 2000 character message length.
- Configured 503 **SUPPORT_NOT_CONFIGURED** response if **SUPPORT_WHATSAPP_NUMBER** environment variable is not defined.
- Logged **SUPPORT_LINK_GENERATED** audit event.

- **Logout (Action 24)**

- Created **POST /api/auth/logout** endpoint for terminating Kamai application sessions.

- Added `revokeSession` to `AuthService` to revoke the specific `RefreshToken` in PostgreSQL matching `bakerId` and `sessionId`.
- Cleared HttpOnly `kamai_access_token` and `kamai_refresh_token` cookies with exact cookie options.
- Logged `USER_LOGGED_OUT` audit log event.
- Added `ADR-008` documenting server-side refresh token revocation and decoupled identity management.

2026-07-25

Added

- **Action 1 — Firebase Phone Authentication**

- `POST /api/auth/firebase/login` — verifies Firebase ID Token, creates/loads baker, issues Kamai session
- `firebase-admin` v14 installed; Firebase Admin SDK plugin (`src/plugins/firebase.ts`)
- `Baker` model: `firebaseUid` (unique, identity key), `phoneNumber` (indexed, non-unique), `status` (`PENDING_ONBOARDING` default), `subscriptionStatus` (`TRIAL` default), nullable profile fields for onboarding
- `RefreshToken` model: SHA-256 hash only (raw token never stored), `id` = `sessionId`, `revokedAt` for future revocation
- `BakerStatus` enum: `PENDING_ONBOARDING`, `ACTIVE`, `SUSPENDED`
- `SubscriptionStatus` enum: `TRIAL`, `ACTIVE`, `EXPIRED`, `CANCELLED`
- Prisma migration: `20260725175131_add_baker_and_refresh_token`
- JWT service using `jose` v5: access token (15m, HS256) + refresh token (7d, HS256)
- JWT payload: `sub` (baker DB id), `firebaseUid`, `phoneNumber`, `sessionId`
- HttpOnly cookies: `kamai_access_token` (15m) + `kamai_refresh_token` (7d), `SameSite=Strict`, `Secure` in production
- `SUSPENDED` baker login blocked with HTTP 403
- `PENDING_ONBOARDING` baker allowed login; `isNewBaker: true` returned so frontend can redirect to onboarding
- Prisma singleton client replacing placeholder

Changed

- `JWT_SECRET`, `JWT_REFRESH_SECRET`, `COOKIE_SECRET` promoted from optional to required in env schema
- `ErrorCode` union: removed `OTP_INVALID`, `OTP_EXPIRED`, `OTP_TOO_MANY_ATTEMPTS`; added `FIREBASE_TOKEN_INVALID`, `FIREBASE_TOKEN_EXPIRED`
- `JwtPayload`: replaced `email` + `role` with `firebaseUid` + `phoneNumber`
- `AuthenticatedUser`: replaced `email` + `role` with `firebaseUid` + `phoneNumber`

Database

- Migration `20260725175131_add_baker_and_refresh_token` applied to Supabase
- `Baker` table created (2 rows pending first login)
- `RefreshToken` table created

Added

- **Project Foundation**
 - Node.js 22 + TypeScript (strict mode) project scaffold
 - Fastify 4.x application factory with plugin architecture
 - Pino structured logger with pretty-printing (dev) and JSON (production)
 - Centralized typed error class hierarchy (`AppError` and subclasses)
 - Environment variable validation using Zod at startup
 - Fastify Helmet plugin (HTTP security headers)
 - Fastify CORS plugin with multi-origin support
 - Fastify Cookie plugin (HttpOnly, Secure, SameSite)
 - Fastify Rate Limiting plugin (IP-based, configurable)
 - Swagger / OpenAPI 3.0 documentation (`/docs`)
 - Health check endpoint (`GET /health`)
 - Prisma ORM scaffold (empty schema, ready for models)
 - Prisma singleton client with query/error logging
 - Shared TypeScript types (`ApiResponse`, `Pagination`, `JWT payload`, `UserRole`)
 - Fastify type augmentation for `req.user`
 - Vitest test framework with coverage (70% threshold)
 - Multi-stage Dockerfile (deps → builder → production, non-root user)
 - Docker Compose (API + Redis with health checks)

- GitHub Actions CI (lint → test → build)
- ESLint + Prettier configuration
- `.env.example` with all anticipated environment variables

Database Changes

- Prisma schema initialized (no tables yet)
- Schema will be populated per action

Security

- HTTP security headers via Helmet
- CORS with credentials support
- IP-based rate limiting
- Sensitive fields redacted from logs (passwords, tokens, cookies)
- Non-root Docker user

Infrastructure

- Docker multi-stage build
- Redis service in Docker Compose
- CI pipeline with PostgreSQL and Redis services